llmctl docs/decision-models

Decision models (Jev-class typed decisions)

Revision: 2 **Last modified:** 2026-10-06T00:00:00Z

llmctl ships six local *decision model* profiles that answer typed questions — `noul` (yes/no probability), `choice` (one option out of N, with per-option probabilities), and `score` (probability-weighted level on an ordered scale) — instead of generating text. This is the same typed- decision paradigm as TypeSafe AI's hosted "Jev / System One" model; llmctl replicates it locally two ways, dispatched per profile by the catalog's `engine` field: stock `llama-server` plus a first-token log-probability readout (the `llama` engine), and a pure-python3 ONNX runner serving encoder-class NLI models (the `onnx` engine: the internal scoring runtime `lib/onnx_server.py`, with the typed-question logic in the Go gateway). Background research: `docs/research/jev-ecosystem.md`, `docs/research/encoder-model-hashes.md`.

Everything on this page describes what exists in the repository today (`cmd/llmctl-decide` + `internal/{gateway,server,contract,readout}`, `lib/decide.sh`, `lib/onnx_server.py`, `models/catalog.json`, `tests/test_decide*.sh`, `tests/test_gateway_endpoints.sh`, `tests/test_onnx_*.sh`), verified against the code — not design intent.

Profiles

profile	model (HF repo / file)	size	min_tier	port	benchmark (with source label)	license
`decide-tiny`	`chaoliangUNSW/Jev-Style-0.8B-Decision-v3-GGUF` / `Jev-Style-0.8B-Decision-v3-Q4_K_M.gguf`	529,296,864 B (~505 MiB)	`below-minimum` (recommends only, never blocks)	8092	79.2% on 2,000 typed decisions per model card — **vendor-measured**; 51 languages	Apache
`decide`	`Mapika/decider-4b-GGUF` / `decider-4b-v2.1-Q4_K_M.gguf`	2,708,804,640 B (~2.52 GiB)	`baseline`	8093	JevBench composite 64.13, ranked #1 of 89 on the JevBench leaderboard — **vendor-measured** (hosted Jev 1.13.0: 63.29, measured by the independent JevBench benchmark repo — **independent**)	Apache
`decide-pro`	`rizzoai-academy/rizzo-flow` / `spark-x2.5-4b-rizzo-`	4,375,021,216 B (~4.07 GiB)	`workstation`	8094	typed-decisions accuracy 0.574 (base) →	Apache

profile	model (HF repo / file)	size	min_tier	port	benchmark (with source label)	license
	`flow-lora-q8_0.gguf`				0.648 (fine-tuned), Q8_0, hosted Jev 0.727 on the same benchmark; Brier 0.205, ECE 0.112 — **vendor-measured** (Rizzo Flow authors, who explicitly disclaim Jev parity and advise recalibration on your own data)	
`decide-nli` (`onnx` engine)	`MoritzLaurer/deberta-v3-large-zeroshot-v2.0` / `onnx/model.onnx` (fp32, opset 12) + `onnx/spm.model`	1,741,985,401 B (~1.62 GiB)	`below-minimum` (recommends only, never blocks)	8096	zeroshot-NLI encoder (DeBERTa-v3-large); no llmctl-run accuracy benchmark — capability claim is the upstream model card's zeroshot-NLI task family — **vendor-measured**	MIT

profile	model (HF repo / file)	size	min_tier	port	benchmark (with source label)	license
					(upstream model card)	
`de-cide-2b`	`alibiserikbay/JevK5-GGUF` / `jevk5-2b-v0.2-Q8_0.gguf`	2,012,012,000 B (~1.87 GiB)	`baseline`	8098	small JevK5 typed-de-cision GGUF; no ll-mctl-run ac-curacy benchmark — **vendor-measured** numbers are on the upstream repo card	Apache
`decide-max`	`alibiserikbay/JevK5-GGUF` / `jevk5-9b-v0.3.3-Q8_0.gguf`	9,527,501,280 B (~8.87 GiB)	`worksta-tion`	8097	largest shipped typed-de-cision GGUF; no ll-mctl-run ac-curacy benchmark — **vendor-measured** numbers are on the upstream repo card	Apache
`decide-julia` (native)	`ggml-org/Ju-lia-1-GGUF` / `Julia-1-Q8_0.gguf`	168,166,496 B (~160 MiB)	`below-minimum`	8103	73.15% (1,463/2,000) on typed de-cisions per	Apache

profile	model (HF repo / file)	size	min_tier	port	benchmark (with source label)	license
					the upstream card (H200 BF16, not the Q8_0 file) — **vendor-measured**; llmctl-run golden numbers (do NOT reproduce the card): Native profiles	
`decide-kev-08b` (native)	`ggml-org/Kev-0.8B-GGUF` / `Kev-0.8B-Q8_0.gguf`	812,406,304 B (~775 MiB)	`below-minimum`	8104	no figure recorded for the upstream card — **unverified**; llmctl-run golden numbers below	Apache
`decide-kev-4b` (native)	`ggml-org/Kev-4B-GGUF` / `Kev-4B-Q4_K_M.gguf`	3,033,489,824 B (~2.82 GiB)	`baseline`	8105	no figure recorded — **unverified**; not run live (scheduler refused on the test host, see below)	Apache
`decide-kev-9b` (native)	`ggml-org/Kev-9B-GGUF` /	6,358,923,744 B (~5.92 GiB)	`workstation`	8106	no figure recorded —	Apache

profile	model (HF repo / file)	size	min_tier	port	benchmark (with source label)	license
	`Kev-9B-Q4_K_M.gguf`				**unverified**; not run live	
`decide-laya` (native)	`ggml-org/Laya-GGUF` / `Laya-Q8_0.gguf`	449,397,600 B (~429 MiB)	`below-minimum`	8107	no figure recorded — **unverified**; llmctl-run golden numbers below	Apache
`decide-lev` (native)	`ggml-org/lev-GGUF` / `lev-Q4_K_M.gguf`	3,011,777,440 B (~2.80 GiB)	`baseline`	8108	68.9% across the 13 S1Bench subsets per the upstream card — **vendor-measured**; not run live	Apache

Eleven profiles are ordinary `llama`-engine catalog entries (five letter-logit, six native `/v1/systemone`) and one (`decide-nli`) is an `onnx`-engine entry; all carry capability `decide`, a pinned `hf_revision` commit, and a pinned per-file sha256 + byte size (catalog `desc` fields record the pin date 2026-10-06 and the override variables `LLMCTL_PORT_<PROFILE>`, `LLMCTL_CTX_<PROFILE>`, `LLMCTL_KVTYPE_<PROFILE>`). `decide-nli` additionally pins its SentencePiece tokenizer (`onnx/spm.model`) and its two small non-LFS configs (`config.json`, which determines the label order, and `tokenizer_config.json`): their sha256 values were computed from the bytes downloaded from huggingface.co and pinned in the catalog (no trust-on-first-use remains). The unused 8.6 MB `tokenizer.json` is no longer downloaded (the runner uses `spm.model`). Every pin was re-verified against huggingface.co (not the mirror) on 2026-10-07; the record is `specs/009-jev-decision-models/evidence/pin-reverify.json`.

Not shipped by default: StartLux-Decision. Its code is Apache-2.0 but its *weights* are **CC BY-NC-4.0** (non-commercial), so it is excluded from the shipped catalog. It remains a user-installable option via a custom profile — add it to your own catalog overlay at your own license risk. llmctl's `decide` mechanism (lettered options + `top_logprobs`) is compatible with any decision-fine-tuned GGUF llama.cpp can load.

How a decision is computed (mechanism)

1. **Prompt rendering (Pattern A, lettered options).** One deterministic prompt (`internal/contract/prompt.go`, the single owner of the template): a fixed header ("You are a decision engine. Read the state and answer the question with exactly one letter."), the `State:` block, `Question: <instructions>`, then one `A) key - description` line per option (letters assigned A..Z in criteria order - JSON object insertion order for `choice`, array order for `score`; `noul` is the fixed pair `A) yes` / `B) no`), and the trailer "Answer with a single letter." / "Answer:". Option-forging text inside the state is neutralised before rendering and the question name is never rendered. Both the gateway and `llmctl decide smoke` use this one template.
2. **One generated token, top-logprobs readout.** The prompt is POSTed to the profile's own `llama-server` (`/v1/chat/completions`, loopback, with the internal engine key) with `max_tokens=1`, `temperature=0`, `logprobs=true`, `top_logprobs` / `n_probs` from the profile's `decision.readout` (default 32), `cache_prompt=false` and - in the default deterministic mode - the fixed seed (`LLMCTL_SEED`, default 1). Repeated requests to one instance are byte-identical.
3. **Letter extraction + renormalization** (`internal/readout`). From the first token's `top_logprobs`, the probability mass of each option letter is summed over its spellings (`"A"`, `" A"`; lower-case `a` / `" a"` and other tokens such as `" the"` do not count); the letters' masses are renormalised with a softmax divided by `LLMCTL_DECIDE_TEMPERATURE` (default 1.0). If the combined letter mass is below the threshold (`LLMCTL_DECIDE_MASS_THRESHOLD`, profile default 0.5) the request fails

with `422 readout_failed` - never a renormalised guess; a letter missing from the top list is flagged, not silently zeroed.

4. **Typed response shaping** (`internal/contract`).
   - `noul` -> `{"type":"noul","noul":<p(yes)>}` - **no confidence key** (Jev 1P wire shape).
   - `choice` -> `{"type":"choice","choice":<argmax key>,"probabilities":{...},"confidence":c}` .
   - `score` -> `{"type":"score","score":<probability-weighted level>,"legend":{...},"probabilities":{...},"confidence":c}` .
   - `confidence = (n*p_max - 1)/(n-1)` clamped to [0,1]. This is the ecosystem convention; **it is not a calibrated error probability** (see Limitations). Probabilities are finite, sum to 1 and are rounded to 9 decimals.
   - `llmctl decide ask` additionally attaches the additive fields `"model": <profile>` and `"evidence": {"profile", "port", "latency_ms"}` . Extra keys are additive; typed-decision SDK clients ignore unknown fields.

Engines are started through the existing scheduler (`llmctl enable` / `auto decide`), so admission control and budget refusals behave exactly as for any other profile - a decision profile that does not fit fails hard with the scheduler's own refusal message. The gateway never starts an engine on demand: a profile with no ready engine answers `503 not_ready` .

Engine dispatch (by decision protocol)

The gateway reads each profile's `decision.protocol` from the catalog and picks the driver:

- **`letter-logit`** (`decide-tiny` , `decide-2b` , `decide` , `decide-pro` , `decide-max` ; engine `llama`): the mechanism above, in `internal/gateway/letter.go` .
- **`nli-onnx`** (`decide-nli` ; engine `onnx`): for each option the gateway sends a premise (the state) / hypothesis pair to the internal encoder runtime's `POST /v1/score` and normalises the entailment column (found

by label NAME, never by position; unusable labels are refused rather than guessed) - `internal/gateway/nli.go`.

- **`systemone-native`** (`decide-julia`, `decide-kev-08b`, `decide-kev-4b`, `decide-kev-9b`, `decide-laya`, `decide-lev`; engine `llama` >= b11379): the engine's native `POST /v1/systemone` (`internal/gateway/native.go`). The gateway re-validates and re-shapes the engine's answer through the contract, and serves these profiles only with `LLMCTL_DECIDE_NATIVE=1` (the engine-advance gate, OD-1); without it they answer `503 not_ready` and the engine is never contacted. See Native profiles.

There is no per-launch engine flag and no proxy hop to the encoder runtime. The typed answer shapes are identical across protocols (`LLMCTL_DECIDE_TEMPERATURE` and `LLMCTL_SEED` apply only to the `letter-logit` driver - the encoder computes entailment probabilities directly, no sampling or logprob scaling is involved).

Native profiles (`systemone-native`, llama.cpp >= b11379)

Six catalog profiles use llama-server's own `POST /v1/systemone` (upstream PR #29818, first tag b11361; b11379 adds the Laya abort fix #29903 and is the pinned engine). Pins (repo, 40-hex revision, size, sha256) are copied from `specs/009-jev-decision-models/evidence/admission/ggml-org__*.json` and re-verified against the Hugging Face tree API at the pinned revision (`evidence/pin-reverify.json`); `tests/test_catalog_json.sh` fails when the catalog drifts from those records. Profile names contain no `.` (the instance separator), hence `decide-kev-08b`.

Launch line (`lib/scheduler.sh`, `sched_build_launch`): the standard decision flags (loopback, per-profile `--api-key-file`, `--parallel 1`, `--no-webui`) plus `--batch-size 4096 --ubatch-size 4096 --no-cache-prompt`. Reason: upstream #30073 - a state longer than the physical batch (default 512) is answered `HTTP 500 "input (N tokens) is too large to process"`; measured with a ~2,100-token state. The gateway maps that exact message (and only on `/v1/systemone`) to `422 validation_failed`; a `501`

`"not a decision model"` (a chat model behind a decision profile) to the non-retryable `500 backend_failed`.

What the gateway does with the native answer: the engine's `model` field is a local file path; the gateway ignores it and answers with the profile name (test: `TestNativeResponseModelIsTheProfileNotTheEnginePath`, live: `determinism-edges.json`). The engine accepts a one-option `choice` with `p = 1.0`; the contract refuses it with `422` before any engine call.

Memory is measured, not estimated. The planner's flat `ctx / 8 MiB` KV rule does not describe these models. Peak RSS of an encoder-class model is dominated by activation buffers that grow with the state length and the window (`evidence/live/ctx-peak-memory-experiment.txt`: Julia-1, 160 MiB of weights, peaked at 1.4 GiB with a 1024-token window and 3.7 GiB with 4096). A profile may therefore declare `defaults.overhead_mb` (planner: added to the RAM footprint in cpu mode, to the VRAM footprint in gpu mode; absent = 0). `decide-julia`, `decide-laya` and `decide-kev-08b` carry a measured value and a window that bounds it (1024 / 1024 / 2048 tokens, so a longer state is answered `422`); `decide-kev-4b`, `decide-kev-9b` and `decide-lev` carry **no measured overhead yet** - they could not be admitted on the test host, so their reservation is weights + the flat KV rule only and is likely an under-estimate. A "cpu mode" (`-ngl 0`) placement is not VRAM-free either: a CUDA build offloads large-batch ops to the GPU (`--op-offload`). On a host with a GPU the planner books that VRAM (measured 0.1-0.2 GiB for Julia-1/Laya, 2.3 GiB for Kev-0.8B, 4.4 GiB for Kev-4B, which is one `nvidia-smi` sample taken 20 s after start with no request sent, not a peak under load: `evidence/live/op-offload-vram-experiment.txt`, `evidence/live/decide-kev-4b/cpu-mode-vram-live-2026-10-08.txt`; `--no-op-offload` removes it but is ~15x slower); a profile whose cpu-mode VRAM is unmeasured books its gpu-mode peak as a ceiling when that is known, else 0 and is reported UNKNOWN (see `docs/hardware-tiers.md`, G-138).

Live results (this host: Ryzen/RTX 3060 12 GB, ~5 GB RAM available, `evidence/live/NATIVE-REPORT.md`): the scheduler admitted `decide-julia` and `decide-laya` (cpu mode); `decide-kev-08b` ran once under the earlier, lower estimate; `decide-kev-4b`, `decide-kev-9b` and `decide-lev` were **refused with numbers** (not forced). Accuracy on the agent-authored golden

set (`docs/golden-set.md` , **human review pending**, 132 items, Wilson 95% intervals, baseline = larger of majority share and chance):

profile	noul n=60	choice n=41	score n=31
`decide-julia`	55.0% [42.5, 66.9], baseline 66.7% - not cleared	34.1% [21.6, 49.5], baseline 23.5% - not cleared	22.6% [11.4, 39.8], baseline 29.0% - not cleared
`decide-laya`	71.7% [59.2, 81.5], baseline 66.7% - not cleared	80.5% [66.0, 89.8], baseline 23.5% - **cleared**	32.3% [18.6, 49.9], baseline 29.0% - not cleared
`decide-kev-08b`	81.7% [70.1, 89.4], baseline 66.7% - **cleared**	87.8% [74.5, 94.7], baseline 23.5% - **cleared**	29.0% [16.1, 46.6], baseline 29.0% - not cleared

Option-order flip rate (share of permutation groups whose answer changes with the option order): Julia-1 41.5% (17/41), Laya 19.5% (8/41), Kev-0.8B 2.4% (1/41). Latency through the HTTPS gateway (golden run, p50 / p95): Julia-1 19 / 32 ms, Laya 61 / 73 ms, Kev-0.8B 192 / 357 ms (cpu mode). The golden set is **agent-authored, human review pending**; these are agreement-with-authored-labels figures, not accuracy claims, and no calibration claim is made.

No calibration claim (fewer than 200 labels); a family or type whose interval lower bound does not exceed its baseline is not "better than trivial" on this set, whatever the point value.

The `onnx` engine: NLI decision semantics and cost model

`lib/onnx_server.py` is the internal encoder scoring runtime: a minimal stdlib HTTP process around `onnxruntime` (Python-only) that scores `(premise, hypothesis)` pairs over loopback; the typed-question semantics below are implemented by the Go gateway on top of it (contract: `specs/009-jev-decision-models/contracts/encoder-runtime.md` , `docs/scripts/onnx_server.md`). `llmctl build onnx` creates a hash-locked private venv (`lib/lock/requirements-onnx.lock` , `--require-hashes`). The

scheduler's `onnx` arm launches it with that venv's python: `onnx_server.py --model-dir ... --host 127.0.0.1 --port ... --profile ... --max-tokens <ctx> --tokenizer <file> --api-key-file <0600 file>`. On Linux it runs under the `llmctl-onnx@.service` systemd template.

Decisions are computed as NLI entailment. The runtime (`POST /v1/score`) answers one FULL softmax row per (premise, hypothesis) pair over `labels` in the model's `id2label` order, plus a `label_source` ; the Go gateway (`internal/gateway/nli.go`) does the typed-question math:

- **premise** = the state text. When the runtime shortened it (opt-in `LLMCTL_DECIDE_TRUNCATE=1` ; refused with 422 otherwise) the response carries `x-llmctl-decide-truncated: true` .
- **hypothesis** = `"<instructions> <option label>"` per option (`yes` / `no` for noul, `key - desc` for choice, the level description for score).
- **label columns are selected BY NAME**, never by position: `entailment` (also `entails` , `entail`) and `contradiction` (`contradicts` , `contradict`) must be present exactly once, case-insensitively; `neutral` is used when present. A **binary** NLI head whose labels are exactly `entailment` and `not_entailment` (also `not-entailment` , `not entailment` , `non_entailment` , `non-entailment`) is accepted as-is: the option score only ever uses P(entailment), and `not_entailment` is its exact complement, so nothing is split into neutral/contradiction or invented. The pinned `decide-nli` model is such a head (`MoritzLaurer/deberta-v3-large-zeroshot-v2.0` , `config.json` id2label `{0: entailment, 1: not_entailment}`). `not_entailment` beside any other label set is refused. A `label_source` starting with `generic-config` or `none` (the model only has `LABEL_n` names), a missing or ambiguous label, a row that is not a probability distribution over `labels` , or the old scalar-per-pair shape is a **configuration error**: the gateway answers `502 backend_failed` with a generic body (the reason goes to the server log only), reports the instance `degraded` in `GET /v1/models` and fails readiness for it for 30 s, then lets one request re-test it. It never guesses a column.

- **option score** = `P(entailment | premise, hypothesis_i)` taken from the entailment column; the question's probabilities are these scores normalised to sum 1:
  - **noul** = `P(entail | yes-hypothesis) / (P(entail | yes-hypothesis) + P(entail | no-hypothesis))`;
  - **choice** = the normalised entailment probability per option;
  - **score** = the same over the levels, then the probability-weighted expectation `sum(i * p_i)`. All-zero entailment is `422 readout_failed`.
- **confidence** (choice/score) = `(n·p_max − 1)/(n−1)` clamped to [0,1] — the same ecosystem-convention formula as the llama path, not a calibrated error probability.

Missing options, budgets and `--permute`. If an option letter is absent from a decoder's first-token alternatives the answer is *flagged* (`flags: ["option_missing"]`, `upper_bounds`): the absent option holds mass, never an exact 0 (the worst case allowed by `min(3 x smallest listed probability, 1 - listed mass)`, jointly at most the unlisted mass; the winner is always a listed option), and `--min-confidence` gates on that worst-case confidence (derivation in `docs/decide-gateway.md`). `llmctl-decide ask --permute K` asks K cyclic option orders and reports the order-averaged answer with `evidence.permute{k,flip_rate}` (choice questions only). A request whose prompt is estimated not to fit the profile's context (`limits.max_context_tokens`, derived from the catalog `ctx`) is a `422` before any completion is requested (the budget of every question is checked first); see `docs/decide-gateway.md`.

Cost model: one encoder forward pass per option. A choice question with n options runs the encoder n times, so latency grows linearly with option count — this is why `decide-nli` ships `ctx: 512` and `parallel: 1`. Inference is CPU-only (onnxruntime `CPUExecutionProvider`), fp32: the planner's footprint branch reserves **RAM = model size × 1.5 + 512 MiB** and **0 VRAM**.

`usage.input_tokens` is an honest estimate — `ceil(input_chars / 4)` over the rendered premise+hypothesis pairs (the ~4-chars-per-token rule of

thumb; SentencePiece token counts are not consulted per request); `usage.output_tokens` is the number of questions.

Laya (BYO-ONNX, not a shipped profile)

`convaiinnovations/laya` (ModernBERT-large + decision head, Apache-2.0) is **not** a shipped catalog profile: the repo ships **no prebuilt ONNX export** (verified 2026-10-06 via the HF API — safetensors only; ONNX is generated client-side by the `laya[onnx]` pip extra, and no `laya-onnx` sibling repo exists on the hub). The runner is deliberately generic, so Laya-class models work via a bring-your-own-ONNX path:

1. Export the model to ONNX (e.g. `pip install "laya[onnx]"` and convert in-process, or any HF Optimum-style export of a sequence-classification head with an NLI-compatible output: 3-class, or binary `entailment` / `not_entailment`).
2. Place `model.onnx` in a model directory together with the tokenizer — `tokenizer.json` (needs `pip install tokenizers`; Laya ships BPE, no `spm.model`) or `spm.model` — and a `config.json` with a real `id2label` naming `entailment` and `contradiction`, or exactly `entailment` and `not_entailment` (a model without one is refused at request time as a label configuration error, see above). Both `<dir>/model.onnx` and `<dir>/onnx/model.onnx` layouts are accepted.
3. Add a custom profile to a **copied** `models/catalog.json` with `"engine": "onnx"`, `"capability": ["decide"]`, a free `"port"`, and `"files"` entries (with real sha256/size) for your export.
4. Point llmctl at it via `LLMCTL_CATALOG=/path/to/catalog.json`, then `llmctl models download <profile>` / `llmctl decide ask --profile <profile>` as usual — no code changes needed.

(`LLMCTL_CATALOG` is the same override the test suites use; editing a copy keeps the shipped catalog pristine.)

Bring your own encoder (FR-008): verified worked example

A USER-ONLY encoder (not in the shipped catalog) was run end to end on 2026-10-09 with `cross-encoder/nli-deberta-v3-xsmall` (Apache-2.0, 286.7 MB, revision `a150876415327c80daeff35ca6f68f5ed8cf5c24`). The only override seam is `LLMCTL_CATALOG`, which **replaces** the whole catalog (there is no merge overlay), so you edit a **copy**. Evidence (captured outputs, sha256 manifest, findings): `specs/009-jev-decision-models/evidence/byo-encoder/`.

```
# 1. copy the shipped catalog and add the profile (fragment: evidence/byo-encoder/overlay-profile-used.json).
#    The port must be unique AND appear both in profiles.<name>.port and in the top-level "ports" map.
cp models/catalog.json ~/llmctl-byo.json
python3 - <<'PY'
import json, os
p = os.path.expanduser("~/llmctl-byo.json")
cat = json.load(open(p))
frag = json.load(open("specs/009-jev-decision-models/evidence/byo-encoder/overlay-profile-used.json"))
cat["profiles"].update(frag["profiles"]); cat["ports"].update(frag["ports"])   # port 18191; change it if taken
json.dump(cat, open(p, "w"), indent=2)
PY
export LLMCTL_CATALOG=~/llmctl-byo.json

llmctl models list | grep byo-nli-xsmall          # listed: onnx, below-minimum, decide
llmctl build onnx                                 # once: the hash-locked private venv (onnxruntime + sentencepiece)
llmctl models download byo-nli-xsmall             # every file sha256-verified, then the built-in onnx smoke
llmctl models verify byo-nli-xsmall               # "passed checksum verification"
llmctl start byo-nli-xsmall                       # the engine (loopback only); or: llmctl enable byo-nli-xsmall
llmctl decide smoke --url http://127.0.0.1:18191 --protocol nli-onnx \
    --key-file "${LLMCTL_STATE_DIR:-$HOME/.local/state/llmctl}/keys/onnx-byo-nli-xsmall.key" --options 3
llmctl decide serve --enable                      # the HTTPS gateway discovers the profile from the catalog
llmctl decide ask --profile byo-nli-xsmall --type choice --state "I was charged twice for March." \
```

```
    --instructions "Which team handles this?" \
    --criteria '{"billing":"invoices, charges,
        refunds","legal":"contracts","support":"technical problems"}'
        --json
```

What the pins must contain: `onnx/model.onnx` (or `model.onnx`), a tokenizer (`spm.model` or `tokenizer.json`), `config.json` with a real `id2label` naming `entailment` and `contradiction`, real `size` + `sha256` per file and an immutable `hf_revision`; `"engine": "onnx"`, `"capability": ["decide"]`, `"decision": {"protocol": "nli-onnx", ...}`. Mark the profile USER-ONLY by keeping it out of `models/catalog.json` (your copy only) and leaving `provenance.benchmark.class` `unverified` and `maturity` / `memory` `unmeasured` until you measure them. `tests/test_catalog_json.sh` logic accepts such an overlay except its repository-maintenance check that every shipped pin has huggingface.co re-verification evidence (not a requirement for your copy).

Verified in the run: overlay accepted, 4 files downloaded and sha256-verified, engine loaded the real ONNX model and answered, `decide smoke` rc 0, the gateway listed the profile and returned a typed `choice` answer (maturity `experimental`, `calibrated:false`). **Not claimed:** decision accuracy for this encoder (unmeasured) and measured RAM/VRAM. Note: on Linux `llmctl start/enable` uses the generated systemd user units, which point at the default state dir, so run with default `LLMCTL_*_DIR` (the evidence run used scratch dirs and therefore started the engine with the argv llmctl generated; see the README there).

Encoder runtime tests (no test seam in production code)

(Historical: the candidate's `LLMCTL_ONNX_FAKE` seam was removed.) The runtime is tested with REAL sockets and the real server code; only the model backends (`onnxruntime`, `sentencepiece`, `tokenizers`) are stub modules under `tests/fixtures/onnx_stubs/`, injected through `PYTHONPATH` by `tests/test_onnx_runtime.sh`, `tests/test_onnx_server.sh` and `tests/test_onnx_download.sh`.

`auto decide` ranking

`sched_rank_for_capability decide` (best-first; `auto decide` picks the first that fits): `decide-nli` > `decide-2b` > `decide` > `decide-pro` > `decide-max` > the six native profiles (`decide-julia` > `decide-laya` > `decide-kev-08b` > `decide-lev` > `decide-kev-4b` > `decide-kev-9b` , ascending footprint, appended last so `auto decide` never prefers an engine-gated profile over a proven one). Rationale per rung (from the comment in `lib/scheduler.sh`): `decide-tiny` (0.5 GiB, calibrated for exactly this workload) > `decide-nli` (1.7 GiB fp32 ONNX encoder; CPU-only but NLI entailment needs no generation, one forward pass per option) > `decide-2b` (1.9 GiB generative, Q8_0 > decide-tiny's Q4_K_M at similar size) > `decide` (2.6 GiB, JevBench #1) > `decide-pro` (4.2 GiB Q8_0, probability-shape tuned) > `decide-max` (9.1 GiB, workstation tier, highest capacity).

Calibration

Fit and apply (FR-080). Collect labelled answers (the winner's probability `p_pred` and whether it was right), run `llmctl-decide calibrate --profile <id> --labels <file> [--method temperature|platt|isotonic]` , and restart or `SIGHUP` the gateway: it then returns a calibrated probability in `confidence` (with `confidence_raw` and `calibration:{method,n,profile_id}`), leaving the raw `probabilities` and the argmax untouched. The profile is bound to the model file's sha256 and to the prompt template hash (which includes `LLMCTL_DECIDE_TEMPERATURE`); a profile fitted for another model, template or temperature is **refused**, never applied. Isotonic needs 1000+ labels, and fewer than 200 labels supports no calibration claim. Details, the worst-case rule for `option_missing` answers and the decision log that collects the labels: `docs/decide-gateway.md` ("Calibration and the decision log"). The calibrator is fitted on the binary "was the winner right" outcome of the winner's own probability - it does not recalibrate the other options' probabilities.

The blunt knob: `LLMCTL_DECIDE_TEMPERATURE` (default `1.0`) divides the letter logprobs before the softmax: values > 1 flatten the distribution (less confident), values < 1 sharpen it (more confident). Non-positive or unparsable

values fall back to 1.0. It is global, not per-model: prefer a fitted calibration profile. Changing it changes the template hash, so profiles fitted at another temperature stop applying until refitted.

sha256 pin procedure

All six shipped profiles carry pinned sha256 + size + `hf_revision` commit hashes in `models/catalog.json` (there is no live-fetch fallback for these LFS weights; the `null` -sha256 escape hatch is only for small non-LFS files, e.g. `decide-nli` 's config files). To re-pin after an intentional upstream change:

```
curl -s "${LLMCTL_HF_BASE:-https://huggingface.co}/api/models/<repo>?
        blobs=true" | python3 -c \
  'import sys,json; d=json.load(sys.stdin); [print(s["rfilename"],
        s["lfs"]["sha256"], s["size"]) for s in d["siblings"] if "lfs"
        in s]'
```

Update the catalog entry's `size` , `sha256` , and `hf_revision` with the new values, then run `llmctl models verify <profile>` against the re-downloaded file. A verification failure against the old pin is the intended signal that upstream re-uploaded — never "fix" it by deleting the pin. The 2026-10-06 pins were captured from the HF API via `hf-mirror.com` ; raw values are archived in `docs/research/decision-model-hashes.md` (iteration-1 GGUF profiles) and `docs/research/encoder-model-hashes.md` (`decide-nli` 's ONNX export + tokenizer, the JevK5 2B/9B GGUFs, and the Laya BYO-ONNX verification).

Live results (nezha, CPU)

> Index of every live run (host, date, tree provenance, validity, headline numbers): `specs/009-jev-decision-models/evidence/live-models/INDEX.md` . Newer pinned nezha runs (2026-10-09, `nezha-pinned-*`) are listed there with their provenance caveats.

Real-model runs on the LAN host `nezha` (i7-1165G7, 8 threads, 64 GB RAM, **CPU only**, `--n-gpu-layers 0`, ctx 8192, llama.cpp `0.5.0-dev` commit `1537a0a`, golden set of 132 originals + 41 option-order permutations, plus 23 probes). Run 2026-10-08. Evidence (copied verbatim, `SHA256SUMS` re-verified locally for every `golden/` and `probes/` directory): `specs/009-jev-decision-models/evidence/live-models/nezha-<profile>-2026-10-08/`.

These nezha numbers are PRE-FIX (they predate the `enable_thinking=false` change in `internal/gateway/letter.go`); the letter-logit profiles' post-fix status is in anton CPU thinking-fix runs (2026-10-09) below.

Provenance caveat: the llmctl tree on nezha was an uncommitted snapshot (no `.git`; the exact source commit is **UNCONFIRMED** — an informal diff suggested it sits between commits `339cb6c` and `f1a22ea`, but no hash-comparison file was captured, so treat that as a lead, not a fact), so these results are **not** pinned to a repository commit and predate HEAD fixes (e.g. `9367d90`). Calibration is "insufficient (n=132, need 200)" on every run; no result below is a calibrated probability.

profile	model (sha256 prefix)	pro-tocol seen	golden well-formed	noul acc [95% CI] (baseline)	choice acc [CI] (baseline)	score acc [CI] (baseline)
decide-lev	lev Q4_K_M (3f61b27c)	syste-mone-native	132/132	0.967 [0.886,0.991] (0.667)	0.976 [0.874,0.996] (0.235)	0.516 [0.348,0.680] (0.290)
decide-kev-4b	Kev-4B Q4_K_M (33ae6b18)	syste-mone-native	132/132	0.950 [0.863,0.983] (0.667)	1.000 [0.914,1.000] (0.235)	0.613 [0.438,0.763] (0.290)
decide-kev-9b	Kev-9B Q4_K_M (86a60849)	syste-mone-native	132/132	0.967 [0.886,0.991] (0.667)	1.000 [0.914,1.000] (0.235)	0.710 [0.534,0.839] (0.290)
decide-pro	spark-x2.5-4b	letter-logit	**68/132**	0.567 [0.441,0.684] (0.667)	0.488 [0.343,0.635] (0.235)	0.290 [0.161,0.466] (0.290)

profile	model (sha256 prefix)	pro-tocol seen	golden well-formed	noul acc [95% CI] (baseline)	choice acc [CI] (baseline)	score acc [CI] (baseline)
	q8_0 (dbec3c89)					
decide (de-cider-4b)	decider-4b-v2.1 Q4_K_M (c7083fcf)	letter-logit	**0/132**	0.000	0.000	0.000
de-cide-2b	jevk5-2b-v0.2 Q8_0 (17222f27)	letter-logit	**0/132**	0.000	0.000	0.000
decide-max	jevk5-9b-v0.3.3 Q8_0 (283de8fd)	n/a	**no res-ult**	-	-	-

What the data shows (and what it does not):

- **decide-lev, decide-kev-4b, decide-kev-9b** ran to completion with every golden request well-formed (0 malformed, 0 HTTP 503/not_ready lines in `golden.log`). For all three the lower CI bound of the `noul`, `choice` and `score` groups exceeds the stated baseline (`lower>baseline=True` in `golden-stats.txt`). The `score` group is the weak one (0.52-0.71). On the 23-request probe set only `noul` clears its baseline for lev/kev-4b/kev-9b; `choice` and `score` probes (n=6, n=4) are too small to separate from baseline. n is small throughout; per-family intervals are wide.
- **decide-pro** (letter-logit) is **PARTIAL / unusable**: only 68 of 132 golden requests (and 11 of 23 probes) were well-formed; the rest returned HTTP 422 `readout_failed` and are counted MALFORMED by `run_golden.py`. Its accuracy over the 132 does not beat the majority baseline on `noul` or `score`. Its `latency.json` covers only the 84 successful golden calls. The 4-option `smoke` failed with `option_missing` (an option letter absent from the first-token alternatives); the 2-option smoke returned rc=0.

- **decide-2b and decide (decider-4b)** produced **0/132 well-formed** (every request HTTP 422 `readout_failed` : "The model produced no usable answer"); their 0.000 accuracies are *malformed-rate artefacts, not model accuracy*. The saved `root-cause-first-tok.txt` probes (max_tokens 1, logprobs) show the engine returning the first token in `reasoning_content` with empty `content` ; the thinking-template cause is now **CONFIRMED live for decide-2b**: after the `enable_thinking=false` fix the same profile went from 0/132 to 128/132 well-formed (anton run 2, below). Pre-fix observation: decide-2b opens with a reasoning preamble ("The"/"Thinking", p~0.78 / ~1.0, option letters absent from the top 8), which a one-token letter-logit readout cannot read. For **decide (decider-4b)** the top-1 first token *is* a letter ("A", logprob -2.301) yet every request was still rejected, so its pre-fix cause was left open here (letter probability mass ~0.12, versus ~0.75 for decide-pro); the post-fix anton diagnosis (below) identifies the rejection criterion as the catalog `mass_threshold` 0.5 against a combined letter mass of 0.44-0.47. decide-pro's first-token alternatives include "We"/"The"/"First" competing with the letters. This is observed on this snapshot only; HEAD later changed `max_options` for JevK5 profiles (`9367d90`) and these two profiles were not re-run on HEAD.
- **No result was invalidated by 503/not_ready**: the only `503|not_ready` matches in the copied logs are a single false hit each in `decide` and `decide-pro` (the strings are `S-022 .. 5036ms` / `C-015~p ..` lines matched by an unrelated digit pattern, not HTTP 503); the failures are 422 readout failures, reported as MALFORMED.
- **decide-max** (JevK5 9B Q8_0): **no result as of 2026-10-08** (`run_all.sh` on nezha, golden phase started 17:19Z, had not finished when the directory was copied); the copied directory is an in-progress snapshot (smoke failed with "no usable answer"; no `RESULT.txt` , no stats). It is **not** a result.
- `RESULT.txt` says `COMPLETED` for every finished profile, including the ones with 0/132 well-formed: it means "the harness finished", not "the model worked". Use `well_formed` and the per-group CIs.
- All numbers are CPU-only (no GPU), single run, uncalibrated; they say nothing about GPU behaviour or about any model not listed.

decide-2b after the thinking fix (anton, CPU) - INCOMPLETE run, not a result

Run 2026-10-08 on host `anton` (CPU only) against `decide-2b` (letter-logit) with the then-uncommitted `chat_template_kwargs.enable_thinking=false` change in `internal/gateway/letter.go` (tree HEAD `c5301de` plus that change, committed later as `5184565` ; `context.txt`). Evidence: `specs/009-jev-decision-models/evidence/live-models/anton-decide-2b-thinkingfix-2026-10-08/` (see its `README.md`). The run was interrupted by a host hang, so it is **incomplete**.

- 58 of 132 golden requests were well-formed (before the fix: 0/132). In `golden.log` 88 requests are `MALFORMED(http 502)` (each after about 27 s), 8 are `MALFORMED(http 422)` and 3 are `MALFORMED(URLError)` .
- Accuracy over the well-formed answers does not beat the baselines (per-group lines in `golden.log` ; for example `ticket_type` n=2 and `weekday_gap` n=2 are far too small to say anything). Calibration reported "insufficient for calibration (n=58, need 200)".
- The cause of the HTTP 502 responses in this run is **UNCONFIRMED**: the gateway returned 502 after about 27 s, but no engine-side evidence in this directory identifies why. The complete run 2 (below) had 0 x 502 for the same profile.
- This is not evidence that the fix works or fails; it shows only that 0/132 became 58/132 on a single, interrupted, CPU-only run. `decide-2b` stays listed as not reliable above.

anton CPU thinking-fix runs (2026-10-09)

Host `anton` (CPU engine, `-ngl 0` , GPU not used), tree HEAD `c5301de` plus the then-uncommitted `internal/gateway/letter.go` (`chat_template_kwargs.enable_thinking=false`) and `resolver.go` changes, now committed as `5184565` (the failing-first run of `TestLetterRequestDisablesThinking` is recorded in `specs/009-jev-decision-models/evidence/g155/failing-first-2026-10-09.txt` : RED on `5184565~1` , GREEN on `5184565`); `build/llmctl-decide` sha256 `0c16f65a...a50f` . Single runs, uncalibrated (n<200), golden 132 originals + 41 permutations and 23 probes. Evidence:

`specs/009-jev-decision-models/evidence/live-models/anton-decide-2b-thinkingfix-run2-2026-10-09/`, `anton-decide-pro-thinkingfix-2026-10-09/`, `anton-decide-thinkingfix-2026-10-09/`, `anton-decide-max-not-exercised-2026-10-09.md` (each README, `golden-stats.txt`, `probes-stats.txt`). Earlier run 1 of decide-2b (above, 58/132, interrupted) is superseded by run 2.

profile	golden well-formed	noul acc [95% CI] (baseline)	choice acc [CI] (baseline)	score acc [CI] (baseline)	probes well-formed	status
de-cide-2b	128/132 (pre-fix 0/132)	0.883 [0.778,0.942] (0.667) beats	0.732 [0.581,0.843] (0.235) beats	0.290 [0.161,0.466] (0.290) **does not** beat	23/23	works for noul choice; score i not shown to b better than ma ity
decide-pro	112/132 (143 x 200, 29 x 502, 1 x 422 over 173 re-cords)	0.867 [0.758,0.931] (0.667) beats	0.659 [0.505,0.784] (0.235) beats	0.645 [0.469,0.789] (0.290) beats	22/23	works, but 502 `backend_fai` on long promp on a CPU eng
decide (de-cider-4b)	smoke 1/3, golden/ probes NOT run	none	none	none	not run	not usable at t current `mass_thresh`
decide-max (9B Q8_0)	NOT exer-cised	none	none	none	not run	no result

- **decide-2b:** option-order flip rate 0.081 (3 of 37 groups); HTTP 200 latency median 2.9 s, max 4.5 s. The 4 malformed golden originals are 8 x HTTP

422 records (C-038..C-041 and permutations), all with `option_count=20` while the catalog `max_options` for decide-2b is 16 (`9367d90`). That is consistent with a gateway max_options refusal, but the response body was not captured, so the reason code is **UNCONFIRMED**. They are not model errors.

- **decide-pro:** 29 x HTTP 502 `backend_failed` on golden (+1 on probes), each about 27 s, `attempts: 3` (the client retries 502 up to `--retries 2` , so about 9 s per attempt). By option count: 12 -> 10, 20 -> 8, 3 -> 3, 8 -> 1, none (noul/score) -> 7. engine.log shows CPU prompt processing at about 21.6 tokens/s (a 168-token prompt took 7.8 s) and `srv stop: cancel task` . A candidate cause exists in the source: the gateway's end-to-end budget `LLMCTL_DECIDE_TIMEOUT` defaults to 8 s (`internal/server/limits.go` , `handlers.go` : a deadline error becomes a 502 with `x-llmctl-decide-reason: deadline_exceeded`). The runs did not override it, but the response headers were not captured, so the 8 s deadline as the cause of these 502s is **INFERRED, not observed**. Accuracy at 12/20 options (0.00, n=5 / n=4) covers only the few answers that finished in time. Option-order flip rate 0.034 (1 of 29 groups); HTTP 200 median 6.6 s.
- **decide (decider-4b Q4_K_M):** smoke 1/3 well-formed; C-001 and C-002 returned HTTP 422 `readout_failed` at 2 options (not a max_options refusal). Diagnosis from captured first-token `top_logprobs` : with the fix the answer letter moves into `content` and its mass rises from 0.12 to 0.44-0.47, but the top-32 holds under half of the mass (diffuse first token), so the combined option-letter mass is below the catalog `mass_threshold` 0.5. The thinking fix works for this model; the model is a diffuse-first-token model for this prompt format. Options not applied: a lower `mass_threshold` with evidence, or a different prompt/readout.
- **decide-max:** not run. The GGUF `jevk5-9b-v0.3.3-Q8_0.gguf` is 8.87 GiB; measured shape on this host is about model + 3 GB resident (expected peak 12-13 GB), above the 10G bounded-run cap. Needs a larger cap with a fresh host-safety review or a GPU with at least 12 GB free.
- All numbers are CPU-only, single run, small n; the malformed items are excluded from accuracy denominators by `stats.py` . The fix is committed (`5184565`); these runs predate that commit and were made on the

uncommitted tree described above, and none was re-run on the committed tree.

decide-nli live result (anton, CPU, wired run)

Run 2026-10-08 on host `anton` (CPU only), the real ONNX encoder `decide-nli` (DeBERTa-v3-large zeroshot, fp32) through the HTTPS gateway, tree HEAD `c5301de` plus the then-uncommitted fix to the live-run harness (committed since as `c00da7b`). Evidence: `specs/009-jev-decision-models/evidence/live-models/decide-nli-main-wiredfix/` (see its `README.md`; `golden/` and `probes/` `SHA256SUMS` re-verified). Single run, uncalibrated (n=132, need 200).

- 132/132 golden and 23/23 probes well-formed; determinism true; option-order flip rate 0 of 41 groups; p50 latency 1543 ms (golden), p95 5552 ms.
- `choice` accuracy 0.829, 95% CI [0.687, 0.915], n=41, **beats** its chance baseline 0.235.
- `noul` 0.533 (majority baseline 0.667) and `score` 0.226 (majority baseline 0.290) do **not** beat their baselines: the NLI encoder is not shown to be good at `noul` or `score`. Use it for `choice`, and treat its `noul` / `score` output as unreliable.
- An earlier anton run (`decide-nli-main-79268f5-INVALID-gateway-miswired`) is invalid and superseded: the harness had not told the gateway the engine address (173 requests got 503).
- This closes the "no real-model evidence" gap for the encoder path only. The letter-logit profiles (`decide`, `decide-2b`, `decide-pro`) failed live in the nezha table above and remain separate open defects.

Honest limitations

- **Accuracy vs hosted Jev.** No shipped local profile matches hosted Jev on knowledge-heavy or multi-hop hard items; the numbers in the profile table are the best available and are labeled vendor-measured or independent per number. Rizzo Flow's authors explicitly disclaim Jev parity (0.648 vs 0.727 on their own typed-decisions benchmark). llmctl makes no parity claims.

- **Option cap.** The lettered-option pattern supports at most 26 options (letters A..Z, hard cap); `LLMCTL_DECIDE_MAX_OPTIONS` (default 20) is the practical cap enforced before any backend call. Larger sets need a grouped runoff — out of scope for v1. `score` supports 2–10 ordered levels.
- **Calibration.** Probability calibration varies per model (the Rizzo authors advise recalibration on your own data). `llmctl-decide calibrate`
  - the gateway's calibration profile (above) is the supported way; without a profile `confidence` stays the uncalibrated shaping convention. A profile is read at start and on `SIGHUP`; its model binding is the catalog's checksum, not a re-hash of the file on disk.
- **Confidence is not an error probability.** `confidence = (n·p_max − 1)/(n−1)` is a shaping convention (1.0 when one option has all the mass, 0.0 at uniform). Do not read it as "probability this answer is correct".
- **`decide capacity` only reports; `decide scale` starts.** `llmctl decide capacity` (and the planner's `decision_instances` subtree) reports how many parallel instances of each decision profile *would* fit — it starts nothing and reserves nothing. `llmctl decide scale <profile> <N>` launches them: admission-bounded (exit 3 with the needed-vs-remaining numbers, nothing started), instance keys `<profile>`, `<profile>.2`, …, registry-allocated ports for every instance beyond the primary, scale-down stops the highest-numbered instance first. See the FAQ for the full behaviour.
- **Native profiles need `LLMCTL_DECIDE_NATIVE=1` and the b11379 engine.** Without the variable the gateway answers `503 not_ready` for them; on an older pinned engine `/v1/systemone` is a 404 and the model does not load. Their working windows are short on purpose (1024 tokens for `decide-julia` and `decide-laya`, 2048 for `decide-kev-08b`): a longer state is a `422`, not a silent truncation (measured: the dense-token and 6,500+ character edge cases in `evidence/live/*/determinism-edges.json`).
- **Question names never reach the model.** In the gateway (`docs/decide-gateway.md`), the `questions` map's *names* are used only to key the `answers` map — matching Jev 1P semantics. All meaning must be in `instructions` / `criteria`; the CLI has no name field at all.

- **logprobs support required.** The mechanism requires a `llama-server` build with OpenAI-compatible logprobs (present in the pinned llama.cpp submodule). If the backend response carries no `choices[0].logprobs.top_logprobs[0]`, the decision fails with an explicit error — the post-download decision smoke test (`_dl_smoke_test_decision` in `lib/download.sh`) is the evidence-based gate that catches this at download time (it runs `llmctl-decide smoke`).

Governance and consumers

Decision models are llmctl's implementation of the decisions-model layer (Constitution §11.4.279; the anchor text is maintained in the constitution, not here). llmctl is a constitution hub engine (§11.4.28(C), §11.4.31; dependency layout); every decision profile is subject to the model validation gate (§11.4.277; validation), with the honest exception that the `decide-*` and colibri suites have been proven against fixtures only. Vision verdicts of HelixQA (§11.4.278) are a separate use of the gate. The consumer integration is claude_toolkit's JEV layer: decision endpoints are registered as `classification: "decision"`, never become aliases, and feed a permission gate, an MCP tool (`jev_ask`) and a launch environment; transport details are in decide-gateway.

Related docs

- `docs/decide-gateway.md` - the Go HTTPS gateway reference.
- `docs/scripts/decide.md` - per-lib internals of the bash front end.
- `docs/user-manual.md` — the `decide` command chapter.
- `docs/hardware-tiers.md` — tier rows and multi-instance capacity semantics.